Roll No: 23

Name: Suthar Devang

MSC AI & ML 3

NLP Practical Assignment

In [ ]:
```
#q1
from pathlib import Path
import re
import nltk
import pandas as pd
from nltk.corpus import gutenberg
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

nltk.download("gutenberg", quiet=True)
documents = []
sources = []

for path in sorted(Path(".").glob("*.csv")):
    frame = pd.read_csv(path)
    columns = frame.select_dtypes(include=["object", "string"]).columns
    if len(columns):
        documents.extend(frame[columns].fillna("").astype(str).agg(" ".join, axis=1
        sources.extend([path.name] * len(frame))

for path in sorted(list(Path(".").glob("*.txt")) + list(Path(".").glob("*.text"))):
    documents.append(path.read_text(encoding="utf-8", errors="ignore"))
    sources.append(path.name)

documents.append(gutenberg.raw("austen-emma.txt")[:50000])
sources.append("NLTK Gutenberg: austen-emma.txt")

def preprocess(text):
    text = re.sub(r"<[^>]+>", " ", text.lower())
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"[^a-z\s]", " ", text)
    tokens = [word for word in text.split() if word not in ENGLISH_STOP_WORDS and l
    return " ".join(tokens)

result = pd.DataFrame({"source": sources, "original": documents, "processed": [prep
preview = result.copy()
preview["original"] = preview["original"].str.slice(0, 120)
preview["processed"] = preview["processed"].str.slice(0, 120)
display(preview.head(10))
print("Documents processed:", len(result))
print("Observations and reasons:")
print("Lowercasing standardizes word forms.")
```

```
print("Removing URLs, HTML, punctuation, and symbols reduces noise.")
print("Removing common stop words reduces low-information terms.")
print("CSV, text files, and an NLTK corpus are processed using the same steps.")
```

	source	original	processed
0	NLTK Gutenberg: austen-emma.txt	[Emma by Jane Austen 1816]\n\nVOLUME I\n\nCHAP...	emma jane austen volume chapter emma woodhouse...

```
Documents processed: 1
Observations and reasons:
Lowercasing standardizes word forms.
Removing URLs, HTML, punctuation, and symbols reduces noise.
Removing common stop words reduces low-information terms.
CSV, text files, and an NLTK corpus are processed using the same steps.
```

q(2) Implement a WordNet lemmatizer and compare its output with stemming using the Gutenberg Corpus.

In [2]:
```
import re
import nltk
import pandas as pd
from nltk.corpus import gutenberg
from nltk.stem import PorterStemmer, WordNetLemmatizer

nltk.download("gutenberg", quiet=True)
nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)

text = gutenberg.raw("austen-emma.txt").lower()
tokens = re.findall(r"[a-z]+", text[:100000])
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
rows = []
seen = set()

for word in tokens:
    stem = stemmer.stem(word)
    lemma = lemmatizer.lemmatize(word)
    if stem != lemma and word not in seen:
        rows.append((word, stem, lemma))
        seen.add(word)
    if len(rows) == 30:
        break

display(pd.DataFrame(rows, columns=["Word", "Stem", "Lemma"]))
```

```
[nltk_data] Integrity check failed for 'omw-1.4': size mismatch (got
[nltk_data]     4070336, expected 26634772)
```

	Word	Stem	Lemma
0	volume	volum	volume
1	woodhouse	woodhous	woodhouse
2	handsome	handsom	handsome
3	comfortable	comfort	comfortable
4	happy	happi	happy
5	disposition	disposit	disposition
6	seemed	seem	seemed
7	unite	unit	unite
8	blessings	bless	blessing
9	existence	exist	existence
10	lived	live	lived
11	nearly	nearli	nearly
12	twenty	twenti	twenty
13	very	veri	very
14	little	littl	little
15	affectionate	affection	affectionate
16	indulgent	indulg	indulgent
17	consequence	consequ	consequence
18	marriage	marriag	marriage
19	his	hi	his
20	house	hous	house
21	early	earli	early
22	died	die	died
23	remembrance	remembr	remembrance
24	supplied	suppli	supplied
25	excellent	excel	excellent
26	as	as	a
27	governess	gover	governess
28	affection	affect	affection
29	family	famili	family

Q(3) Implement the Bag of Words model using the 20 Newsgroups dataset.

In [3]:
```
import pandas as pd
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer

dataset = fetch_20newsgroups(subset="train", remove=("headers", "footers", "quotes"
vectorizer = CountVectorizer(stop_words="english", max_features=10000)
X = vectorizer.fit_transform(dataset.data)
features = vectorizer.get_feature_names_out()
print("Document-feature matrix shape:", X.shape)
display(pd.DataFrame(X[:5, :20].toarray(), columns=features[:20]))
```

```
Document-feature matrix shape: (11314, 10000)
```

	00	000	005	01	02	03	030	04	040	05	06	07	08	09	0b	0c	0d	0g	0h	0i
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Q(4) Implement TF-IDF, Bag of Words, and One-Hot Encoding from scratch using NumPy only.

In [4]:
```
import numpy as np

documents = ["the cat sat on the mat", "the dog sat on the log", "cat and dog are f
tokens = [document.lower().split() for document in documents]
vocabulary = sorted(set(word for document in tokens for word in document))
word_index = {word: index for index, word in enumerate(vocabulary)}

bow = np.zeros((len(documents), len(vocabulary)), dtype=int)
for row, document in enumerate(tokens):
    for word in document:
        bow[row, word_index[word]] += 1

one_hot = {word: np.eye(len(vocabulary), dtype=int)[index] for word, index in word_
one_hot_documents = [np.array([one_hot[word] for word in document]) for document in
lengths = np.array([len(document) for document in tokens]).reshape(-1, 1)
tf = bow / np.maximum(lengths, 1)
document_frequency = np.count_nonzero(bow, axis=0)
idf = np.log((len(documents) + 1) / (document_frequency + 1)) + 1
tfidf = tf * idf

print("Vocabulary:", vocabulary)
print("Bag of Words:")
print(bow)
print("One-Hot Encoding for the first document:")
print(one_hot_documents[0])
```

```
print("TF-IDF:")
print(np.round(tfidf, 3))
```

```
Vocabulary: ['and', 'are', 'cat', 'dog', 'friends', 'log', 'mat', 'on', 'sat', 'th
e']
Bag of Words:
[[0 0 1 0 0 0 1 1 1 2]
 [0 0 0 1 0 1 0 1 1 2]
 [1 1 1 1 1 0 0 0 0 0]]
One-Hot Encoding for the first document:
[[0 0 0 0 0 0 0 0 0 1]
 [0 0 1 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 1 0]
 [0 0 0 0 0 0 0 1 0 0]
 [0 0 0 0 0 0 0 0 0 1]
 [0 0 0 0 0 0 1 0 0 0]]
TF-IDF:
[[0.    0.    0.215 0.    0.    0.    0.282 0.215 0.215 0.429]
 [0.    0.    0.    0.215 0.    0.282 0.    0.215 0.215 0.429]
 [0.339 0.339 0.258 0.258 0.339 0.    0.    0.    0.    0.   ]]
```

Q(5) Implement TF-IDF vectorization on the IMDB movie reviews dataset and extract important features.

In [ ]:
```
import numpy as np
import pandas as pd
from tensorflow.keras.datasets import imdb
from sklearn.feature_extraction.text import TfidfVectorizer

(x_train, y_train), _ = imdb.load_data(num_words=10000)
word_index = imdb.get_word_index()
index_word = {index + 3: word for word, index in word_index.items()}
index_word.update({0: "<PAD>", 1: "<START>", 2: "<UNK>", 3: "<UNUSED>"})
texts = [" ".join(index_word.get(index, "<UNK>") for index in review) for review in

vectorizer = TfidfVectorizer(max_features=10000, min_df=2)
X = vectorizer.fit_transform(texts)
features = vectorizer.get_feature_names_out()
print("TF-IDF matrix shape:", X.shape)
row = X[0].toarray().ravel()
top_indices = np.argsort(row)[-10:][::-1]
display(pd.DataFrame({"Feature": features[top_indices], "TF-IDF": row[top_indices]})
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/im
db.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/im
db_word_index.json
1641221/1641221 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
TF-IDF matrix shape: (25000, 9772)
```

	Feature	TF-IDF
0	was	0.281024
1	the	0.269068
2	brilliant	0.223690
3	amazing	0.219642
4	film	0.169077
5	and	0.165555
6	children	0.146750
7	just	0.132556
8	they	0.130607
9	profile	0.129449

Q(6) Perform positive and negative sentiment analysis and compare three machine learning models, including relevant preprocessing.

In [6]:
```
import re
import nltk
import pandas as pd
from nltk.corpus import movie_reviews
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

nltk.download("movie_reviews", quiet=True)

def preprocess(text):
    text = re.sub(r"<[^>]+>", " ", text.lower())
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"[^a-z\s']", " ", text)
    return re.sub(r"\s+", " ", text).strip()

fileids = movie_reviews.fileids()
texts = [movie_reviews.raw(fileid) for fileid in fileids]
labels = [movie_reviews.categories(fileid)[0] for fileid in fileids]
X_train_text, X_test_text, y_train, y_test = train_test_split(
    texts, labels, test_size=0.2, random_state=42, stratify=labels
)
vectorizer = TfidfVectorizer(preprocessor=preprocess, ngram_range=(1, 2), max_featu
X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)
models = {
    "Multinomial Naive Bayes": MultinomialNB(),
    "Logistic Regression": LogisticRegression(max_iter=1000),
```

```
    "Linear SVM": LinearSVC()
}
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(y_test, predictions, pos_label="pos", zero_div
        "Recall": recall_score(y_test, predictions, pos_label="pos", zero_division=
        "F1-score": f1_score(y_test, predictions, pos_label="pos", zero_division=0)
    })

display(pd.DataFrame(results).round(3).sort_values("F1-score", ascending=False))
```

	Model	Accuracy	Precision	Recall	F1-score
2	Linear SVM	0.868	0.845	0.900	0.872
1	Logistic Regression	0.828	0.805	0.865	0.834
0	Multinomial Naive Bayes	0.832	0.845	0.815	0.830

Q(7) Find the edit distance between two strings and display the dynamic programming table as a pandas DataFrame.

In [7]:
```
import pandas as pd

s1 = input("Enter the first string: ")
s2 = input("Enter the second string: ")
m, n = len(s1), len(s2)
dp = [[0] * (n + 1) for _ in range(m + 1)]

for i in range(m + 1):
    dp[i][0] = i
for j in range(n + 1):
    dp[0][j] = j

for i in range(1, m + 1):
    for j in range(1, n + 1):
        cost = 0 if s1[i - 1] == s2[j - 1] else 1
        dp[i][j] = min(
            dp[i - 1][j] + 1,
            dp[i][j - 1] + 1,
            dp[i - 1][j - 1] + cost
        )

display(pd.DataFrame(dp, index=["Ø"] + list(s1), columns=["Ø"] + list(s2)))
print("Edit distance:", dp[m][n])
```

	∅	H	a	p	p	y
∅	0	1	2	3	4	5
H	1	0	1	2	3	4
e	2	1	1	2	3	4
l	3	2	2	2	3	4
l	4	3	3	3	3	4
o	5	4	4	4	4	4

```
Edit distance: 4
```

Q(8) Build unigram, bigram, and trigram language models from scratch, store probabilities in dictionaries, predict the next word, and compare perplexity.

In [8]:
```
import re
import numpy as np
import nltk
from collections import Counter
from nltk.corpus import gutenberg
import pandas as pd

nltk.download("gutenberg", quiet=True)
sentences = [
    re.findall(r"[a-z]+(?:'[a-z]+)?", " ".join(sentence).lower())
    for sentence in gutenberg.sents("austen-emma.txt")
]
sentences = [sentence for sentence in sentences if sentence]
split_at = int(len(sentences) * 0.8)
train_sentences = sentences[:split_at]
test_sentences = sentences[split_at:]
word_counts = Counter(word for sentence in train_sentences for word in sentence)
common_words = {word for word, count in word_counts.most_common(5000)}
vocabulary = sorted(common_words | {"<UNK>", "</s>"})
vocabulary_set = set(vocabulary)
vocab_size = len(vocabulary)
alpha = 1.0
counts = {n: Counter() for n in (1, 2, 3)}
context_counts = {n: Counter() for n in (2, 3)}

def normalize_sentence(sentence):
    return [word if word in vocabulary_set else "<UNK>" for word in sentence]

for sentence in train_sentences:
    words = normalize_sentence(sentence)
    for n in (1, 2, 3):
        sequence = ["<s>"] * (n - 1) + words + ["</s>"]
        for i in range(n - 1, len(sequence)):
            gram = tuple(sequence[i - n + 1:i + 1]) if n > 1 else (sequence[i],)
            counts[n][gram] += 1
            if n > 1:
                context_counts[n][gram[:-1]] += 1
```

```
probabilities = {1: {}, 2: {}, 3: {}}
unigram_total = sum(counts[1].values())
for word in vocabulary:
    probabilities[1][(word,)] = (counts[1].get((word,), 0) + alpha) / (unigram_tota
for n in (2, 3):
    for gram, count in counts[n].items():
        probabilities[n][gram] = (count + alpha) / (context_counts[n][gram[:-1]] +

def word_probability(n, context, word):
    if n == 1:
        return probabilities[1].get((word,), alpha / (unigram_total + alpha * vocab
    context = tuple(context[-(n - 1):])
    gram = context + (word,)
    if gram in probabilities[n]:
        return probabilities[n][gram]
    return (counts[n].get(gram, 0) + alpha) / (context_counts[n].get(context, 0) +

def predict_next(prefix, n):
    context = normalize_sentence(re.findall(r"[a-z]+(?:'[a-z]+)?", prefix.lower()))
    candidates = [word for word in vocabulary if word not in {"<UNK>", "</s>"}]
    return max(candidates, key=lambda word: word_probability(n, context, word))

def perplexity(test_data, n):
    log_probabilities = []
    for sentence in test_data:
        words = normalize_sentence(sentence) + ["</s>"]
        history = ["<s>"] * (n - 1)
        for word in words:
            log_probabilities.append(np.log(word_probability(n, history, word)))
            if n > 1:
                history = (history + [word])[-(n - 1):]
    return float(np.exp(-np.mean(log_probabilities)))

results = pd.DataFrame({
    "N-gram Model": ["Unigram", "Bigram", "Trigram"],
    "Perplexity": [perplexity(test_sentences, n) for n in (1, 2, 3)],
    "Stored Probabilities": [len(probabilities[n]) for n in (1, 2, 3)]
})
display(results.round(3))
for n in (1, 2, 3):
    print(f"{n}-gram next-word prediction:", predict_next("she was", n))
```

	N-gram Model	Perplexity	Stored Probabilities
0	Unigram	394.743	5002
1	Bigram	805.578	51027
2	Trigram	2862.246	99005

```
1-gram next-word prediction: the
2-gram next-word prediction: not
3-gram next-word prediction: not
```

Q(9) Implement Naive Bayes from scratch using an NLTK corpus, handle out-of-vocabulary words and smoothing, and compare it with the library Naive Bayes model using classification metrics.

In [9]:
```
import re
import nltk
import numpy as np
import pandas as pd
from nltk.corpus import movie_reviews
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

nltk.download("movie_reviews", quiet=True)

def preprocess(text):
    text = re.sub(r"<[^>]+>", " ", text.lower())
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"[^a-z\s']", " ", text)
    return re.sub(r"\s+", " ", text).strip()

fileids = movie_reviews.fileids()
texts = [movie_reviews.raw(fileid) for fileid in fileids]
labels = np.array([movie_reviews.categories(fileid)[0] for fileid in fileids])
X_train_text, X_test_text, y_train, y_test = train_test_split(
    texts, labels, test_size=0.2, random_state=42, stratify=labels
)
vectorizer = CountVectorizer(preprocessor=preprocess, max_features=20000, min_df=2)
X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)
classes = np.unique(y_train)
alpha = 1.0
feature_log_prob = {}
class_log_prior = {}

for label in classes:
    mask = y_train == label
    label_matrix = X_train[mask]
    token_counts = np.asarray(label_matrix.sum(axis=0)).ravel()
    feature_log_prob[label] = np.log(token_counts + alpha) - np.log(token_counts.su
    class_log_prior[label] = np.log(mask.sum() / len(y_train))

scores = np.column_stack([
    np.asarray(X_test @ feature_log_prob[label]).ravel() + class_log_prior[label]
    for label in classes
])
custom_predictions = classes[np.argmax(scores, axis=1)]
library_model = MultinomialNB(alpha=1.0)
library_model.fit(X_train, y_train)
library_predictions = library_model.predict(X_test)

def evaluate(name, predictions):
    return {
```

```
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(y_test, predictions, pos_label="pos", zero_div
        "Recall": recall_score(y_test, predictions, pos_label="pos", zero_division=
        "F1-score": f1_score(y_test, predictions, pos_label="pos", zero_division=0)
    }

results = [
    evaluate("Naive Bayes from scratch", custom_predictions),
    evaluate("Scikit-learn MultinomialNB", library_predictions)
]
display(pd.DataFrame(results).round(3))
unknown_example = vectorizer.transform(["zzunknownwordxyz qqqnewtermabc"])
print("Features found for unseen words:", unknown_example.nnz)
print("Vocabulary size:", X_train.shape[1])
```

	Model	Accuracy	Precision	Recall	F1-score
0	Naive Bayes from scratch	0.81	0.81	0.81	0.81
1	Scikit-learn MultinomialNB	0.81	0.81	0.81	0.81

```
Features found for unseen words: 0
Vocabulary size: 20000
```